

המכללה האקדמית להנדסה ע"ש סמי שמעון (SCE)

המחלקה להנדסת חשמל ואלקטרוניקה

פרויקט קורס פייתון "שכחה קטסטרופלית"

מרצה: אנדרי דולגין

מתרגל: דמיטרי בחובסקי

מגישים:

תומר נור מילקוב 325996577

יהד אור 317927051

המאמר המרכזי: "Overcoming catastrophic forgetting in neural networks"

לינק: <https://arxiv.org/abs/1612.00796>

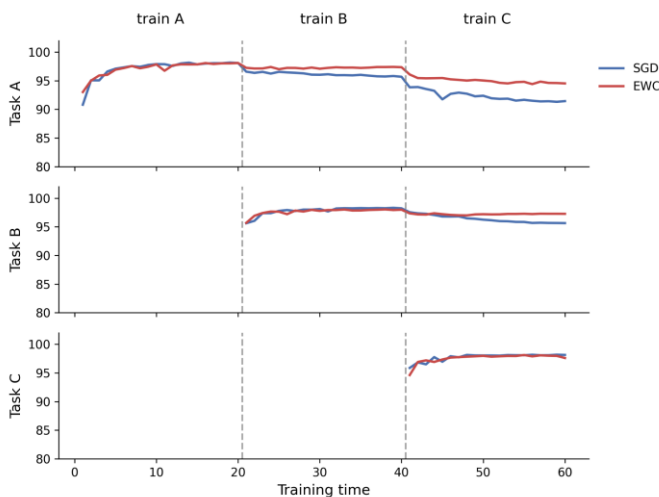
נושא הפרויקט:

שחזור תוצאות מתוך מאמר בתחום **למידה מתמשכת** (**Continual Learning**), המתמקד בבעיה של **Catastrophic Forgetting** ובפתרון בשם **Elastic Weight Consolidation (EWC)**.

המאמר: "Overcoming catastrophic forgetting in neural networks" - המאמר עוסק בבעיה מרכזית ברשתות נוירונים. רשת נוירונים - מודל חישובי בהשראת המוח האנושי, המהווה את הבסיס ללמידה עמוקה (Deep Learning).

הבעיה: כאשר רשת נוירונים לומדת משימה חדשה, היא שוכחת משימות קודמות. זה קורה כי אותן משקלות מתעדכנות שוב ושוב.
הפתרון: הרעיון המרכזי: לא כל המשקולות זהות באותה מידה.
EWC מזהה אילו משקולות חשובות למשימות קודמות ומגביל שינוי שלהן בזמן למידה חדשה. דבר זה מאפשר לרשת ללמוד כמה משימות ברצף מבלי לשכוח את הישגות.

הגרפים ששחזרנו בפרויקט:



גרף A - למידה לאורך זמן:

ביצועים של הרשת לאורך זמן עבור 3 משימות (A, B, C). כחול - SGD רגיל. אדום - EWC.

SGD - שוכח משימות קודמות.
EWC - שומר על ביצועים.

EWC מונע catastrophic forgetting.

גרף B - ביצועים מול מספר משימות:

X: מספר משימות.

Y: דיוק.

כחול - SGD.

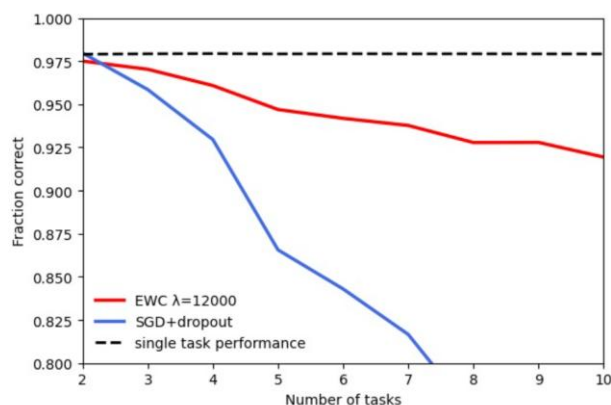
אדום - EWC.

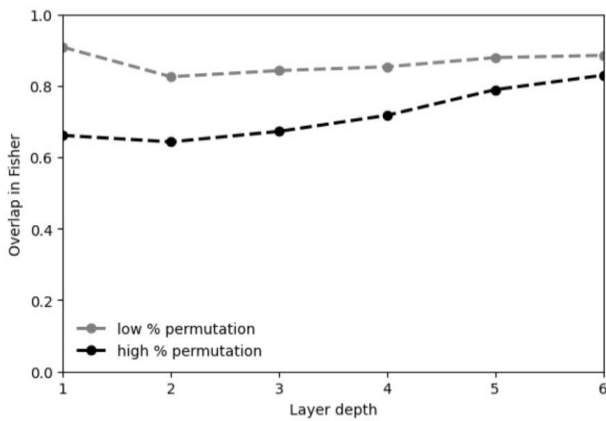
שחור מקווקו - ביצוע אידיאלי (משימה בודדת).

SGD יורד חזק

EWC נשאר כמעט קבוע

ככל שיש יותר משימות - SGD שוכח יותר, EWC מצליח לשמור על ידע.





גרף C - חפיפה בין משימות:

X: עומק שכבה.

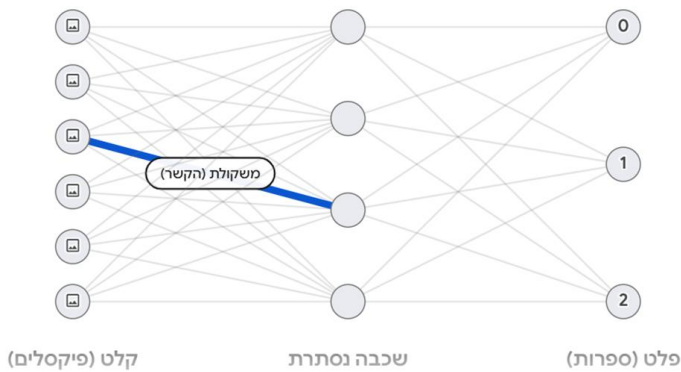
Y: Fisher overlap (דמיון בין משימות).

אפור - שינוי קטן.

שחור - שינוי גדול.

שינוי קטן - דמיון גבוה.

שינוי גדול - דמיון נמוך בהתחלה, אבל עולה עם עומק הרשת.



שכבות מוקדמות:

- לומדות תכונות בסיסיות (פיקסלים).
- רגישות לשינוי - overlap נמוך.

שכבות עמוקות:

- לומדות ייצוגים מופשטים.
- פחות רגישות - overlap עולה.

הדמיון בין משימות תלוי גם בכמה הן שונות וגם באיזה עומק ברשת מסתכלים.

תהליך עבודה ותוצאות

מטרת הפרויקט:

לשחזר את הגרפים A, B, C באמצעות:

- Python + PyTorch.
- שימוש ב-AI לבניית קוד וניתוח.

שלב 1 - הבנת המאמר:

קריאה והבנת:

- EWC
- Fisher Information
- Permuted MNIST

והכנת הפרמטרים לפי סביבת העבודה.

שלב 2 - הכנת סביבת עבודה:

התקנת:

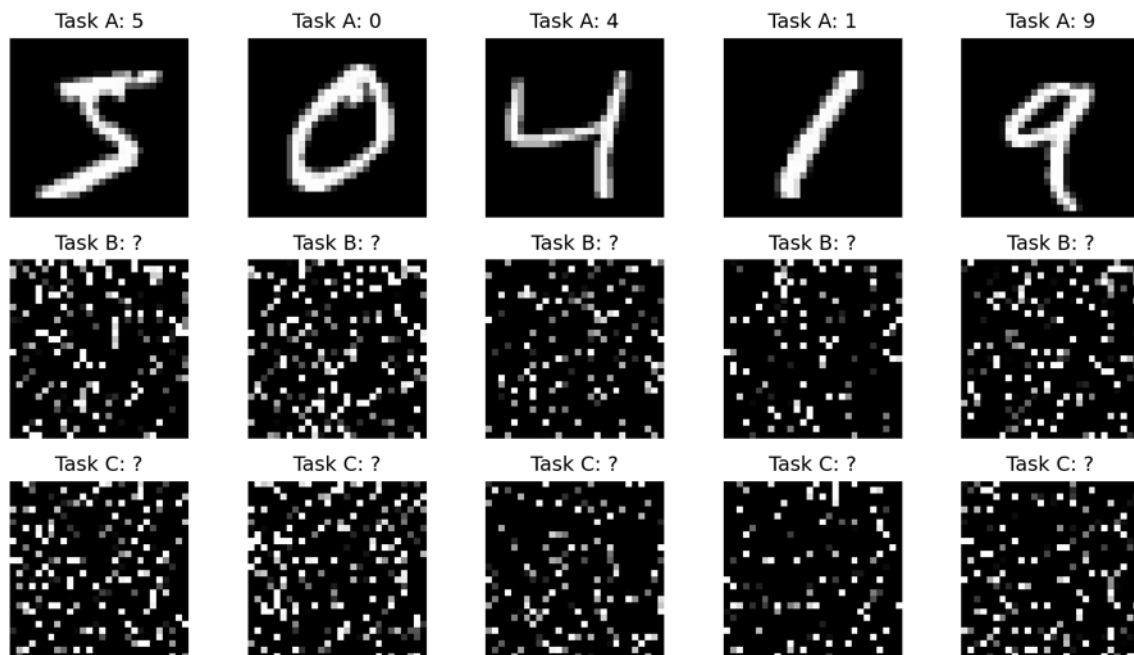
- Python
- PyTorch - היא ספריית למידת מכונה / Deep Learning שרצה בתוך Python. משתמשים בה כדי לבנות מודלים, לאמן רשתות נוירונים ועוד.
- torchvision - ספרייה משלימה של PyTorch לטובת עבודה עם תמונות.

עבודה ב:

- Google Colab / PC

שלב 3 - מימוש Permuted MNIST:

Permuted MNIST: How the Network "Sees" the Tasks



מסד הנתונים (MNIST (Modified National Institute of Standards and Technology הוא אוסף קלאסי של 70,000 תמונות קטנות (28x28 פיקסלים) בגווני אפור של ספרות בכתב יד (0-9). הוא משמש בעיקר כסטנדרט ללימוד, בדיקה ואימון של אלגוריתמים ללמידת מכונה וראייה ממוחשבת, וכולל 60,000 תמונות לאימון ו-10,000 לבדיקה.

במאמר שבחרנו ישנו מסד נתונים **Permuted MNIST** הוא גרסה משונה של ערכת הנתונים המקורית MNIST (תמונות של ספרות בכתב יד). הוא משמש בעיקר כ-benchmark (מדד ביצועים) בתחום של למידה מתמשכת (Continual Learning) (ובינה מלאכותית כדי לבחון יכולת הסתגלות של מודלים).

- יצירת משימות (tasks) - ערבוב הפיקסלים אקראית.

שלב 4 - מימוש רשת נוירונים:

בניית מודל חישובי המורכב משכבות של "נוירונים" מלאכותיים, הלומדים לזהות תבניות בנתונים.

שלב 5 - מימוש SGD רגיל:

מימוש SGD (Stochastic Gradient Descent) הוא תהליך למידה המבוסס על עדכון פרמטרים של מודל (כמו משקולות ברשת נוירונים) על סמך דוגמה בודדת בכל צעד, במקום על כל נתוני האימון בבת אחת.

נראה לאחר מימוש שהמודל שוכח את המשימה הקודמת עם כל משימה באה שהוא לומד.

שלב 6 - מימוש EWC:

להגביל את השינוי במשקולות שהיו חשובות למשימות הקודמות, תוך מתן אפשרות למשקולות פחות חשובות להתעדכן עבור המשימה החדשה.

המודל מזהה אילו משקולות היו הכי קריטיות למשימה A והקוד יוסיף להם קנס מתמטי כבד לפונקציית ההפסד (Loss function) בזמן הלמידה.

אם משקולת לא הייתה חשובה, הקוד יאפשר לשנות אותו בקלות כדי ללמוד את משימה B.

איך הקוד יודע מה חשוב?

בעזרת מושג שנקרא "מטריצת פישר" (Fisher Information Matrix). בקוד שלנו, זהו פשוט מערך (Array) באותו גודל של המשקולות (לפי שכבות הנוירונים שנבחר), ששומר מספר עבור כל משקולת: מספר גדול = משקולת קריטית (לא לגעת!), מספר קרוב לאפס = משקולת פנויה לשינויים. במקרה שלנו, שכבות הנוירונים מזהות תבניות של כל מספר מ-0-9 (קווים ישרים, מעגלים, אלכסונים וכו') ונותנות משקל לכל פיקסל בתבנית מספר מ-0-9.

שלב 7 - מימוש Online EWC (בונוס):

הבעיה: אלגוריתם EWC רגיל שומר "רשימת מנעולים" (מטריצת פישר) נפרדת בזיכרון המחשב עבור כל משימה שלמדנו. כשהמודל נדרש ללמוד 10, 50 או 100 משימות ברצף, זיכרון המחשב קורס מעומס, וזמן החישוב של קנסות פונקציית ההפסד הופך לאיטי מדי (Penalty Overload).

הפתרון: Online EWC פותר את בעיית הזיכרון על ידי שמירת מטריצת פישר אחת ויחידה לאורך כל התוכנית.

במקום לייצר מערכים חדשים בזכרון, הקוד משתמש בשיטת "ממוצע נע" (Running Average). בסיום משימה חדשה, הקוד מעדכן את המטריצה **היחידה** על ידי חיבור של הזיכרון הישן) למשל 50% מהערך השמור) יחד עם החשיבות של המשימה החדשה (50%). כך, המחשב שומר על ידע מכל המשימות הקודמות בתוך משתנה קוד אחד בלבד, שומר על יעילות הזיכרון ומאפשר סקיילינג (Scaling) לאינסוף משימות.

שלב 8 - שחזור גרפים סופי

כתבנו קוד מתאים, שחזרנו את הגרפים במלואם וראינו כי EWC מהווה פתרון של שכחה קטסטרופלית.

דו"ח שימוש ב-AI בפרויקט:

מטרת השימוש ב-AI:

בפרויקט נעשה שימוש במערכת בינה מלאכותית ככלי עזר למחקר, פיתוח וניתוח תוצאות.

מטרת השימוש הייתה:

- חקר ראשוני להבנת המאמר ומטרתו.
- בניית קוד ב-Python וב-PyTorch.
- שחזור הגרפים מהמאמר.
- ניתוח תוצאות.
- התאמת פרמטרים (Hyperparameters).
- זיהוי פערים בין תוצאות השחזור לתוצאות המאמר.

לימוד והבנת המאמר:

בשלב הראשון נעשה שימוש ב-AI לצורך:

- הסבר מושגי Catastrophic Forgetting.
- הסבר Elastic Weight Consolidation (EWC).
- הסבר Fisher Information Matrix.
- ניתוח המשמעות של גרפים A, B ו-C.

יצירת הקוד הראשוני:

- מודל רשת נוירונים.
- יצירת משימות Permuted MNIST.
- חישוב Fisher Information.
- מימוש אלגוריתם EWC.
- יצירת גרפים לצורך השוואה מול המאמר.
- הקוד שנוצר עבר התאמות ושיפורים לאורך הפרויקט.

שיפור התוצאות:

אחד השימושים המרכזיים ב-AI היה תהליך איטרטיבי של ניתוח תוצאות. לאחר כל הרצה נאספו:

- ערכי Fisher overlap.
- דיוקי המודל.
- גרפים שנוצרו בפועל.
- עיצוב אסטרטגיית ממוצע הפישר הרץ ב-Online EWC.

נתונים אלו הועברו ל-AI לצורך ניתוח.

התאמת Hyperparameters:

בהתבסס על הפלטים והגרפים, נעשה שימוש ב-AI לצורך:

- שינוי מספר השכבות.
- שינוי רוחב הרשת.
- שינוי מספר epochs.
- שינוי מספר seeds.
- שינוי גודל קבוצת Fisher.
- שינוי אופן חישוב Fisher overlap.

לאחר כל שינוי בוצעה הרצה חדשה ונבדקה מידת ההתאמה למאמר.

אימות מול המאמר:

- לאמת את פירוש הגרפים שקיבלנו.
- לוודא שהקוד מיישם את אותן משימות כמו במאמר.
- לבדוק האם גרף מסוים באמת קרוב לגרף המקורי, ואם לא, למה.
- לזהות מצבים בהם השינויים שבוצעו אינם משפרים את ההתאמה.